Inhalt der Checkliste

Inhalt der Checkliste **1**
Projektabgabe Baby Tools World **2**
1. Repository 2
.gitignore Datei 2
README.md 2
Django-Code 3
requirements.txt 3
2. Code-Style 4
3. Dokumentation 4
4. Hinweise 4

Projekt Abgabe - Baby Tools World

Dieses Dokument enthält eine Checkliste mit den wichtigsten Punkten zu den Projektanforderungen. Dabei werden die Anforderungen in mehrere Bereiche/Kategorien aufgeteilt, um den Einstieg in das Projekt ein wenig zu vereinfachen.

Bei deiner Abgabe solltest du uns unbedingt die Credentials für deinen Superuser mitschicken, sodass wir uns auch einloggen und in deiner Baby Tools World umschauen können
→ Die Zugangsdaten als **Kommentar bei der Abgabe** im Campus mitschicken, **nicht** im Repository speichern

Für Code-Projekte solltest du immer folgendermaßen vorgehen:

1. Lege dir einen Fork unseres Repository in deinen Github Namespace an
2. Entferne die Fork-Eigenschaft deines Repositories
3. Erzeuge einen Feature-Branch `add-product-tags` für die Implementierung in deinem Repository
4. Auf deinem Feature-Branch erbringst du alle Entwicklungsleistungen. Du solltest für den Feature-Branch einen **Pull-Request** anlegen, den du uns für ein Code-Review zusenden kannst. Der Feature-Branch soll dabei solange erhalten bleiben, bis die Mentoren das Projekt und damit den Feature-Branch zum Merge freigeben.

1. Repository

.gitignore

- ☑ ~~Die gitignore Datei ignoriert alle File-Patterns, die für Python Projekte üblicherweise ignoriert werden~~
- ☑ ~~Achte darauf, die virtuelle Umgebung auch als Teil der gitignore aufzunehmen~~
- ☑ ~~Achte darauf, dass die .gitignore Datei auch Umgebungsvariablen (`.env` Dateien) ignoriert. Dabei ist es egal ob die Datei `.env`, `local.env`, `development.env`, oder Ähnlich heißt.~~
  - ☑ ~~Die einzige Ausnahme von dieser Regel ist die Datei `example.env`~~

README.md

- ☑ ~~Die README beginnt mit einer Überschrift vom Typ H1 -> Repository/Projekt Name~~
- ☑ ~~Die README enthält direkt zu Beginn eine Beschreibung des Repositories, ohne dabei zu technisch zu werden. Diese Beschreibung sollte nicht mehr als 2-5 Sätze umfassen.~~
- ☑ ~~Eine Sektion "Quickstart" sollte als Teil der README enthalten sein. Hier sollen kurz die Voraussetzungen genannt und eine Schnellstart-Anleitung beschrieben sein.~~

- [x] ~~Es soll eine Sektion "Usage" geben, in der eine erweiterte Benutzung, Konfiguration der Anwendung beschrieben wird~~

Django Code

- [x] ~~`products/models.py`:~~
  - [x] ~~Erweitere die models.py so, dass dort ein Model Names `Tag` existiert, welches folgende Attribute enthält:~~
    - [x] ~~name~~
    - [x] ~~id (wird automatisch von django vergeben)~~
    - [x] ~~created_at (soll automatisch erzeugt werden) — **diese Eigenschaft ist read-only**~~
    - [x] ~~updated_at (soll automatisch erzeugt werden) — **diese Eigenschaft ist read-only**~~
  - [x] ~~Erweitere das `Product` Model so, dass dort eine weitere Eigenschaft `tags` existiert. Diese Tags sollen eine ManyToMany Verknüpfung zu den `Tags` über die ID haben~~
    - [x] ~~`tags` können beim `Product` nicht gesetzt/leer sein, d.h. tags sind nicht zwingend für Produkte notwendig~~

- [x] ~~products/templates/product.html (Django Template Language)~~
  - [x] ~~Erweitere das product.html so, dass unter der Rating Summary die Product Tags gerendert werden~~
    - [x] ~~In der Produktdetails Seite sollen auf der Karte des Produkts unten über dem "Kaufen" Button die Tags angezeigt werden, die mit einem Produkt verknüpft sind~~
    - [x] ~~über den Tags soll eine Überschrift mit dem label "Product Tags" angezeigt werden~~
    - [x] ~~Falls ein Produkt **keine** tags hat, soll in der entsprechenden Sektion die Überschrift angezeigt werden und darunter ein Label "no tags available"~~

- [x] ~~products/admin.py~~
  - [x] ~~Erweitere das Admin Template so, dass das Tag Model über die Oberfläche von Django sichtbar ist, bzw. im Admin Panel neue Tag Objekte angelegt bzw. verwaltet werden können~~
    - [x] ~~Importiere die models.tag Klasse~~
    - [x] ~~Registriere das Tag Model~~
    - [x] ~~Erweitere das ProductAdmin mit einem Filter~~

- [x] ~~products/views.py~~
  - [x] ~~Erweitere die views.py so, dass folgendes gilt:~~
    - [x] ~~Nach dem erfolgreichen Abschicken einer Bewertung sollen alle Formularfelder (Rating-Stars und Kommentar-Text) geleert werden~~

requirements.txt

- [x] ~~Alle Paketabhängigkeiten wurden in der requirements.txt erfasst~~
- [x] ~~Die requirements.txt enthält nur Pakete mit einer gepinnten Version~~

2. Code-Style

- [x] ~~Der Python Code sollte PEP 8 als Code-Style Standard implementieren, dazu kannst du ein Linting Tool verwenden wie bspw. flake8. Mehr Informationen:~~
  - [x] ~~PEP 8 Style-Guide for Python https://peps.python.org/pep-0008/~~
  - [x] ~~Flake 8 Dokumentation https://flake8.pycqa.org/en/latest/~~
- [x] ~~Folgende Namenskonventionen solltest du berücksichtigen:~~
  - [x] ~~Klassennamen -> PascalCase~~
  - [x] ~~Variablen -> snake_case~~
- [x] ~~Deine Python Funktionen sollten type-hints für Funktionsparameter, Rückgabe-Werte, und Variablen verwenden~~
- [x] ~~Dein Python Code sollte Doc-Strings an jeder Funktion enthalten, die beschreiben, was die Funktion tut, welche Parameter sie erhalten kann, bzw. welchen Return Value man erwarten kann.~~

3. Dokumentation

Allgemeines

In dieser Sektion findest du Hinweise hinsichtlich der erwarteten Dokumentation deines Projektes. Die folgenden Punkte sollen dir helfen, bei deiner Dokumentation möglichst nichts zu vergessen, was es zu beachten gilt.

- [x] ~~Die Sprache der Dokumentation (README/Code-Comments) ist **Englisch**~~
- [x] ~~Die Dokumentation sollte in vollständigen Sätzen formuliert sein, zum Beispiel in der README-Datei eines Repositories~~
  - [x] ~~Bei längeren Dokumentationen wird ein eigenständiger Ordner ***docs*** im Repository angelegt, der die Dokumentation enthält.~~

4. Hinweise

Allgemeine Hinweise

- [x] ~~Du solltest dein Projekt mit einer virtuellen Umgebung aufsetzen und betreiben~~
- [x] ~~Bei Fragen oder Unklarheiten komm' gern auf uns zu und wende dich mit dem Ticketsystem an uns.~~
- [x] ~~Du kannst die offizielle Dokumentation zu Django oder Python jederzeit verwenden. Solltest du Fragen zur Dokumentation haben achte darauf, uns einen Link zur Doku mitzuschicken.~~
  - [x] ~~Django Dokumentation: https://docs.djangoproject.com/en/5.2/~~
  - [x] ~~Python Dokumentation: https://docs.python.org/3/~~
- [x] ~~Du solltest vor der Abgabe deine Anwendung mindestens einmal bei dir gestartet und alles getestet haben -> also auf deinem Computer~~
- [x] ~~Zusätzlich zu deinem GitHub Repository solltest du ein kurzes Loom Video (maximal 5min.) aufnehmen und bereitstellen, indem du kurz deine Abgabe zeigst und vorstellst was du getan hast - dabei musst du nicht alle Details erwähnen, jedoch sollst du auf alle relevanten Schritte kurz eingehen und diese zeigen.~~
- [x] ~~Die Migrationen sind nach deinen Änderungen erstellt und angewendet~~

Testing

Bevor Du dein Projekt einreichst, solltest du die folgenden Dinge sicherstellen und getestet haben:

- [x] ~~Die Baby Tools World ist erreichbar unter der Adresse `<DEINE_VM_IP>:8000`~~
- [x] ~~Die Applikation zeigt dir Produkte und Kategorien an~~
  - [x] ~~Du kannst auf einer Produktseite mehr Infos zu einem Produkt, sowie dessen Tag sehen.~~
- [x] ~~Du kannst über das Admin-Panel neue Produkte, Kategorien, sowie Tags anlegen.~~
- [x] ~~Das Kommentarfeld ist nach dem Absenden geleert~~
- [x] ~~Du kannst dein Container Image mit Hilfe des Befehls in der README erzeugen~~
- [x] ~~Du kannst den Container mit Hilfe der README starten und die Applikation wieder im Browser erreichen.~~